

# ASC 选拔题目报告（选择大题：Embodied World Model-WMA）

## 一、基本信息

| 项目     | 内容                                                                                                                                      |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 姓名     | 王文博                                                                                                                                     |
| 年级专业   | 2024级 计算机科学与技术 绿色算力创新班                                                                                                                  |
| 大题     | Embodied World Model (UnifoLM-WMA-0)                                                                                                    |
| 大题仓库链接 | <a href="https://github.com/Wangwenbo2006/ASC-WMA-FINALPROGRAM-REPORT">https://github.com/Wangwenbo2006/ASC-WMA-FINALPROGRAM-REPORT</a> |

## 二、题目理解

- 选择题目：Embodied World Model (UnifoLM-WMA-0)
- 题目链接：<https://github.com/unitreerobotics/unifolm-world-model-action>
- 赛题仓库：<https://github.com/ASC-Competition/ASC26-Embodied-World-Model-Optimization>
- 大题主要内容：运行具身智能世界模型推理，生成视频，并通过 PSNR 评估生成质量。目标是在保证输出质量满足要求的前提下，尽可能减少程序运行时间。
- 性能目标：在保证 PSNR 不低于 Baseline 或尽量接近的前提下，降低推理时间。
- 正确性判断方式：PSNR 结果、输出视频是否完整、是否存在黑屏/花屏/帧数异常。

## 三、机器环境

| 项目     | 内容                                                                           |
|--------|------------------------------------------------------------------------------|
| 机器来源   | AutoDL 云服务器（GPU 实例）                                                          |
| 操作系统   | Ubuntu 22.04.1 LTS                                                           |
| CPU    | 8 vCPU（Intel Xeon）                                                           |
| 内存     | 754 GB                                                                       |
| GPU    | NVIDIA RTX 4090 (24 GB)                                                      |
| Python | 3.10.20                                                                      |
| CUDA   | 12.1（PyTorch 内置）                                                             |
| 关键依赖版本 | PyTorch 2.3.1, transformers 4.40.1, diffusers 0.30.2, open-clip-torch 2.22.0 |

## 四、Baseline

### 4.1 Baseline 运行命令

```
cd /root/autodl-tmp/unifolm-world-model-action
{ time bash ASC26-Embodied-World-Model-Optimization-
main/unitree_g1_pack_camera/case1/run_world_model_interaction.sh; } 2>&1 | tee output_asc.log
```

### 4.2 Baseline 配置

| 配置项            | 值                      |
|----------------|------------------------|
| 场景             | unitree_g1_pack_camera |
| Case           | case1                  |
| ddim_steps     | 50                     |
| FP16           | 否                      |
| guidance_scale | 1.0                    |
| frame_stride   | 6                      |
| n_iter         | 11                     |

4.3 Baseline 结果

| 项目    | 内容             |
|-------|----------------|
| 运行时间  | 22m 1.837s     |
| PSNR  | 18.34 dB       |
| 正确性结果 | ✅ 输出视频完整，无异常   |
| 输出文件  | 0_full_fs6.mp4 |
| 日志文件  | output_asc.log |

五、性能记录方法

本次使用以下性能分析方法：

- ✅ `time` 命令（记录总运行时间）
- ✅ `nvidia-smi`（实时观察 GPU 利用率和显存）
- ✅ 定时 GPU 监控（`watch -n 1 nvidia-smi`）
- ✅ 程序日志分析（每步耗时记录）
- ✅ 各实验独立日志文件（`output.log`）

实际执行的命令格式：

```
cd /root/autodl-tmp/unifolm-world-model-action && \
{ time CUDA_VISIBLE_DEVICES=0 python3 scripts/evaluation/world_model_interaction.py \
  --seed 123 \
  --ckpt_path /root/autodl-tmp/unifolm-world-model-action/checkpoints/UnifolM-WMA-0-Dual/unifolm_wma_dual.ckpt \
  --config configs/inference/world_model_interaction.yaml \
  --savedir [实验输出目录] \
  --bs 1 --height 320 --width 512 \
  --unconditional_guidance_scale [guidance_scale] \
  --ddim_steps [ddim_steps] \
  --ddim_eta [ddim_eta] \
  --prompt_dir unitree_g1_pack_camera/case1/world_model_interaction_prompts \
  --dataset unitree_g1_pack_camera \
  --video_length 16 \
  --frame_stride [frame_stride] \
  --n_action_steps 16 \
  --exe_steps 16 \
  --n_iter 11 \
  --timestep_spacing uniform_trailing \
  --guidance_rescale 0.7 \
  --perframe_ae ; } 2>&1 | tee [输出目录]/output.log
```

六、性能现象

根据工具输出，记录到以下明确现象：

| 观察对象           | 现象                    | 数据或证据                                    |
|----------------|-----------------------|------------------------------------------|
| Baseline 总运行时间 | 22m 1.837s            | real 22m1.833s                           |
| GPU 利用率（推理阶段）  | 60-80%，去噪循环中接近满载      | nvidia-smi 观察                            |
| 显存使用           | 约 18-20GB，接近 24GB 上限  | nvidia-smi 观察                            |
| 每步耗时（50步）      | 约 110 秒/步，基本稳定        | 110.61s/it                               |
| 每步耗时（30步+FP16） | 约 46 秒/步，显著下降         | 46.04s/it                                |
| CLIP 模型加载      | 加载 3.9GB CLIP 模型，仅需一次 | 日志可见 Loading pretrained ViT-H-14 weights |

七、瓶颈判断

| 项目          | 内容                                                              |
|-------------|-----------------------------------------------------------------|
| 判断的瓶颈类型     | GPU 计算瓶颈（DDIM 采样步数）                                             |
| 主要证据        | ① 每步耗时稳定，与 DDIM 步数直接相关；② GPU 在去噪循环中利用率接近满载；③ 减少 DDIM 步数后，时间线性下降 |
| 为什么优先处理这个瓶颈 | DDIM 采样步数直接控制模型计算量，每减少一步都能线性节省时间                                |
| 可能存在的其他瓶颈   | ① 模型加载阶段（17GB 权重 + 3.9GB CLIP）；② 数据预处理；③ 视频编码输出                 |

八、优化方法与实验

8.1 优化实验设计

本次优化围绕多个方向展开：

1. **DDIM 采样步数优化**: 减少去噪迭代次数
2. **FP16 混合精度推理**: 利用 Tensor Core 加速
3. **torch.compile**: 尝试 JIT 编译优化
4. **引导尺度调整**: 优化生成质量
5. **帧步长调整**: 改变采样密度

## 8.2 关键优化方法说明

### 方法一：DDIM 步数优化

| 项目         | 内容                                                          |
|------------|-------------------------------------------------------------|
| 优化方法名称     | DDIM 采样步数调整                                                 |
| 修改位置       | 命令行参数 <code>--ddim_steps</code>                             |
| 修改前行为      | <code>--ddim_steps 50</code> (Baseline)                     |
| 修改后行为      | <code>--ddim_steps 30</code> 或 <code>--ddim_steps 20</code> |
| 预期改善的瓶颈    | 减少去噪循环次数，降低 GPU 计算量                                         |
| 可能影响正确性的风险 | 步数过少可能导致采样未收敛，画质下降                                          |

### 方法二：FP16 混合精度推理

| 项目         | 内容                                                                                            |
|------------|-----------------------------------------------------------------------------------------------|
| 优化方法名称     | FP16 自动混合精度                                                                                   |
| 修改位置       | <code>scripts/evaluation/world_model_interaction.py</code> 中的 <code>run_inference()</code> 函数 |
| 修改前行为      | 模型以 FP32 精度推理                                                                                 |
| 修改后行为      | 模型前向传播在 <code>autocast</code> 上下文中以 FP16 执行                                                   |
| 预期改善的瓶颈    | 利用 RTX 4090 Tensor Core 加速矩阵运算                                                                |
| 可能影响正确性的风险 | 数值精度降低可能导致 PSNR 轻微下降                                                                          |

### 方法三：torch.compile

| 项目      | 内容                                                                                |
|---------|-----------------------------------------------------------------------------------|
| 优化方法名称  | torch.compile JIT 编译                                                              |
| 修改位置    | <code>model = torch.compile(model, mode="reduce-overhead", fullgraph=True)</code> |
| 预期改善的瓶颈 | 消除 Python 开销, 融合算子                                                                |

## 九、对照实验

所有实验均在 同一台 AutoDL 服务器、同一块 RTX 4090 GPU、同一个场景 (unitree\_g1\_pack\_camera/case1) 下运行, 确保结果可比性。

## 9.1 完整实验数据汇总

| 实验编号 | 优化方法 | ddim_steps | FP16 | frame_stride | guidance_scale | 运行<br>时间 | 加速比<br>(vs<br>Baseline) | PSNR<br>(dB) | 结论 |
|------|------|------------|------|--------------|----------------|----------|-------------------------|--------------|----|
|------|------|------------|------|--------------|----------------|----------|-------------------------|--------------|----|

|          |                       |     |   |   |     |           |        |       |                           |
|----------|-----------------------|-----|---|---|-----|-----------|--------|-------|---------------------------|
| Baseline | 无                     | 50  | ✗ | 6 | 1.0 | 22m 01.8s | 1.00x  | 18.34 | 基准                        |
| 实验1.1    | 减少步数                  | 30  | ✗ | 6 | 1.0 | 15m 15.5s | 1.44x  | 17.07 | ✅可行，轻微质量下降                |
| 实验1.2    | 减少步数                  | 20  | ✗ | 6 | 1.0 | 9m 54.9s  | 2.22x  | 6.39  | ✗质量崩溃，不可用                 |
| 实验2.1    | FP16                  | 30  | ✅ | 6 | 1.0 | 9m 47.0s  | 2.25x  | 17.24 | ✅良好平衡                     |
| 实验2.2    | FP16                  | 20  | ✅ | 6 | 1.0 | 6m 56.2s  | 3.17x  | 6.39  | ✗质量崩溃                     |
| 实验2.3    | FP16                  | 50  | ✅ | 6 | 1.0 | 7m 34.4s  | 2.91x  | 18.40 | ✅最优方案                     |
| 实验2.4    | FP16                  | 100 | ✅ | 6 | 1.0 | 12m 26.2s | 1.77x  | 17.13 | ⚠步数过多，PSNR 下降             |
| 实验2.5    | FP16 + 高引导尺度          | 50  | ✅ | 6 | 3.0 | ~7m 18s   | ~3.02x | 18.40 | ⚠guidance_sc 无影响          |
| 实验3.1    | FP16 + torch.compile  | 30  | ✅ | 6 | 1.0 | 10m 34.8s | 2.08x  | 17.24 | ⚠未加速，增加编译开销               |
| 实验4a     | FP16 + 引导尺度 2.0       | 30  | ✅ | 6 | 2.0 | 10m 06s   | 2.18x  | 17.24 | ⚠无提升                      |
| 实验4b     | FP16 + 引导尺度 3.0       | 30  | ✅ | 6 | 3.0 | 9m 54s    | 2.23x  | 17.24 | ⚠无提升                      |
| 实验2.8    | FP16 + frame_stride=4 | 50  | ✅ | 4 | 1.0 | 7m 21s    | 3.00x  | 17.05 | ⚠frame_stride 小，PSNR 下降   |
| 实验2.9    | FP16 + frame_stride=8 | 50  | ✅ | 8 | 1.0 | 7m 22s    | 2.99x  | 15.22 | ⚠frame_stride 大，PSNR 严重下降 |

9.2 关键优化结果对比

| 对比项  | Baseline  | 实验2.3（最优） | 变化       |
|------|-----------|-----------|----------|
| 运行时间 | 22m 01.8s | 7m 34.4s  | ↓ 65.6%  |
| 加速比  | 1.00x     | 2.91x     | ↑ 2.91 倍 |
| PSNR | 18.34 dB  | 18.40 dB  | +0.06 dB |

十、正确性验证

| 检查项      | Baseline | 实验2.3（最优） | 是否通过 |
|----------|----------|-----------|------|
| 程序是否正常结束 | ✅        | ✅         | ✅    |
| 是否生成完整视频 | ✅        | ✅         | ✅    |

|             |          |          |   |
|-------------|----------|----------|---|
| 视频是否存在明显异常  | 无异常      | 无异常      | ✓ |
| PSNR        | 18.34 dB | 18.40 dB | ✓ |
| PSNR 是否明显下降 | —        | 基本持平（略升） | ✓ |
| 是否修改输入或评价方式 | —        | 否        | ✓ |

**正确性说明：**

实验2.3（FP16 + ddim\_steps=50）的 PSNR 为 18.40 dB，与 Baseline（18.34 dB）基本持平，甚至略高 0.06 dB，属于正常数值波动。输出视频完整，无黑屏、花屏或帧数异常，生成质量保持稳定。

## 十一、结果分析

### 1. 优化是否带来了推理时间下降？

是的。最优方案（实验2.3：FP16 + ddim\_steps=50）将推理时间从 22m 01.8s 降至 7m 34.4s，**时间下降 65.6%，加速比达到 2.91x。**

### 2. 如果变快，主要原因是什么？

主要有两个因素叠加：

- **FP16 混合精度：**利用 RTX 4090 的 Tensor Core 加速矩阵运算，是主要的加速来源。
- **配合合理的步数：**保持 50 步保证了质量，同时 FP16 大幅缩短了每步耗时（从 110s/步降至 32s/步）。

### 3. 为什么 ddim\_steps=20 质量崩溃？

20 步去噪时采样轨迹未充分收敛，扩散模型的逆过程过早截断，导致生成帧中出现大量高频噪声和结构失真。该模型需要至少 30 步才能保证基础视觉质量。

### 4. 为什么增加步数至 100 反而 PSNR 下降？

100 步时模型可能**过度去噪**，导致生成结果偏离参考视频。FP16 精度误差在长步数下也可能累积，影响质量。50 步是当前的最优

### 5. torch.compile 为何未加速？

- **首次编译开销：**消耗了额外的运行时间。
- **动态形状问题：**模型内部可能存在动态张量操作，导致图优化效果打折。
- **FP16 已接近硬件极限：**额外优化空间有限。

### 6. guidance\_scale 调整为何无效？

实验4a（2.0）和4b（3.0）的 PSNR 均与实验2.1（17.24 dB）完全一致，说明该模型在当前场景下对引导尺度不敏感。

### 7. frame\_stride 调整为何无效？

- frame\_stride=4（更密集采样）：PSNR 17.05 dB（下降）
- frame\_stride=8（更稀疏采样）：PSNR 15.22 dB（严重下降）
- 默认 frame\_stride=6 是最佳值

### 8. 本次实验还有哪些控制不够严格的地方？

- 未进行多次运行取平均值，单次运行可能受系统负载影响。
- 未使用 PyTorch Profiler 获取 kernel 级时间分布，瓶颈判断主要基于步数变化和外推。
- torch.compile 实验未区分首次编译时间和后续稳定运行时间。

## 十二、总结

### 12.1 作业完成情况

本次作业成功完成了以下工作：

- 1. **Baseline 确认**：在 RTX 4090 上运行 UnifoLM-WMA-0 Baseline，记录推理时间 22m 01.8s，PSNR 18.34 dB。
- 2. **性能记录与分析**：通过 `time` 命令、`nvidia-smi` 监控和程序日志分析，确认 **DDIM 采样步数** 和 **FP16 混合精度** 是影响性能的关键因素。
- 3. **优化实验**：共完成 **12 组** 优化实验，涵盖 DDIM 步数调整、FP16 混合精度、`torch.compile`、引导尺度调整、帧步长调整等方向。
- 4. **正确性验证**：所有有效优化方案的输出视频均正常，PSNR 在可接受范围内。

12.2 有效优化与无效优化

有效优化：

-  **FP16 混合精度推理**（实验2.3）：加速 2.91 倍，PSNR 保持 18.40 dB
-  **DDIM 步数调整至 30**（实验2.1）：加速 2.25 倍，PSNR 17.24 dB（可接受）
-  **DDIM 步数调整至 50 + FP16**（实验2.3）：最优方案

无效/失败优化：

-  `ddim_steps=20`（实验1.2/2.2）：PSNR 崩溃至 6.39 dB
-  `torch.compile`（实验3.1）：未加速，有编译开销
-  `guidance_scale` 调整（实验4a/4b）：无影响
-  `frame_stride` 调整（实验2.8/2.9）：PSNR 下降

12.3 主要瓶颈判断

通过逐项实验验证，确认：

- **DDIM 采样步数**：主要性能瓶颈，直接影响计算量
- **FP16 混合精度**：有效加速手段，利用 Tensor Core
- **其他参数**（`guidance_scale`、`frame_stride`）：对性能影响有限

12.4 优化结论

最佳生产配置： `FP16 + ddim_steps=50`

| 指标   | 数值              | 对比 Baseline    |
|------|-----------------|----------------|
| 推理时间 | <b>7m 34.4s</b> | 下降 65.6%       |
| 加速比  | <b>2.91x</b>    | 提升 2.91 倍      |
| PSNR | <b>18.40 dB</b> | 持平（略升 0.06 dB） |

12.5 后续改进方向

- 1. **更高效采样器**：尝试 DPM-Solver，在更少步数下获得更高质量，可能突破 PSNR 瓶颈
- 2. **模型部署优化**：使用 TensorRT 或 ONNX Runtime 进行推理加速
- 3. **缓存机制**：避免重复加载模型和 CLIP 编码器
- 4. **异步 I/O**：对视频写入进行异步处理，减少等待
- 5. **跨场景验证**：在更多场景上验证优化效果

附录：实验数据文件列表

| 实验编号  | 日志文件               | PSNR 文件           | 输出视频           |
|-------|--------------------|-------------------|----------------|
| 实验1.1 | output_steps30.log | psnr_steps30.json | 0_full_fs6.mp4 |

|       |                                   |                                  |                |
|-------|-----------------------------------|----------------------------------|----------------|
| 实验1.2 | output_steps20.log                | psnr_steps20.json                | 0_full_fs6.mp4 |
| 实验2.1 | output_fp16_steps30.log           | psnr_fp16_steps30.json           | 0_full_fs6.mp4 |
| 实验2.2 | output_fp16_steps20.log           | psnr_fp16_steps20.json           | 0_full_fs6.mp4 |
| 实验2.3 | output.log                        | psnr.json                        | 0_full_fs6.mp4 |
| 实验2.4 | output.log                        | psnr.json                        | 0_full_fs6.mp4 |
| 实验2.5 | output.log                        | psnr.json                        | 0_full_fs6.mp4 |
| 实验2.8 | output.log                        | psnr.json                        | 0_full_fs4.mp4 |
| 实验2.9 | output.log                        | psnr.json                        | 0_full_fs8.mp4 |
| 实验3.1 | output_compile_fp16_steps30.log   | psnr_compile_fp16_steps30.json   | 0_full_fs6.mp4 |
| 实验4a  | output_guidance2_fp16_steps30.log | psnr_guidance2_fp16_steps30.json | 0_full_fs6.mp4 |
| 实验4b  | output_guidance3_fp16_steps30.log | psnr_guidance3_fp16_steps30.json | 0_full_fs6.mp4 |

报告结束